Source Code Archive README

Bulk RNA-seq analysis of zebrafish (Danio rerio) flank skeletal muscle, comparing telomerase-mutant and wild-type fish across age. This archive contains the analysis programs, their inputs and outputs, and instructions for running them.

Archive Contents

Sr.No.	File	Description
1	differential_expression_analysis.py	Differential expression pipeline for Q1 (genotype), Q2 (age) and Q3 (overlap), plus QC and figures.
2	q2_age_outlier_sensitivity.py	Robustness check: re-runs the Q2 (age) contrast with and without the three PCA-flagged outlier samples and compares the results.
3	go_enrichment_analysis.py	Functional characterisation of the DEGs: GO/ pathway enrichment (g:Profiler), transcription-factor analysis, gene correlation, detailed Q3 overlap, and threshold exploration.
4	tert_3month_de_analysis.py	DE analysis of the 3-month (young) dataset: builds the count matrix from 13 featureCounts files, runs the MUT/Het contrasts, and optionally compares to the 18-month result.
5	cross_dataset_direction_check.py	Cross dataset direction check: locates the 18-month Q1 DEGs in the 3-month data and tests whether they show early directional drift (means, one-sample t-tests, Mann-Whitney vs background, and an 18mo-vs-3mo fold-change correlation).
6	grn_tf_activity_model.py	GRN/transcription-factor activity model: maps zebrafish genes to human orthologues (g:Profiler), scores curated CollecTRI regulons via decoupler to infer TF activities, and exports a reusable network (edge list + GraphML).
7	string_prep_and_grn_overlap.py	STRING input preparation and GRN cross-reference: turns the Q1 DEGs into the gene lists submitted to STRING (mapped to human orthologues, split up/down), and cross-references STRING's enrichment tables against the GRN transcription-factor target sets.
8	q2_age_tf_activity_and_comparison.py	Natural ageing TF activity and full Q1-vs-Q2 comparison: scores all CollecTRI transcription factors against the Q2 age contrast from scratch, then compares the full mutant (Q1) and ageing (Q2) Tf-activity tables (not the 71 TF-subset).
9	interaction_genotype_x_age.py	Genotype x Age interaction model with per-gene classification: fits the 2x2 (genotype x age 3mo/18mo) DESeq2 interaction model and classifies every gene by whether it is

		affected by genotype, age, both or neither, and how the genotype effect changes with age.
10	grn_interaction_pattern_mapping.py	Maps the GRN onto the interaction-model patterns: tests (Fisher exact) whether the GRN target genes are enriched for the age-dependent-onset pattern, and summarises per TF how its targets behave.
11	project_overview_plots.py	Project overview figures: the sample design, the number of significant transcription factors per contrast, and a dashboard of the main results across all analyses.
	README document	This file as the "table of contents and usage instructions" for the archive.

Programs

1. differential_expression_analysis.py

Purpose: End-to-end differential expression (DE) analysis addressing three questions:

- **Q1 – Genotype effect:** 18-month mutant vs 18-month wild-type (M18_MUT vs M18_WT).
- **Q2 – Age effect:** 37-month vs 11-month wild-type (M37_WT vs M11_WT).
- **Q3 – Overlap:** genes significant in both Q1 and Q2, and whether they move in the same or opposite direction.

Input: gene_count.xls – tab-delimited raw read-count matrix for the 18-month adult-muscle experiment. Contains a gene_id column, 20 sample columns (M18_MUT_1 … M11_WT_5), and gene-annotation columns (gene_name, gene_chr, gene_biotype, Family, etc.). Sample groups: M18_MUT (n=4), M18_WT (n=5), M37_WT (n=6), M11_WT (n=5). Set path at the top of the script via COUNT_MATRIX_PATH.

Method Summary: Low-count genes are filtered, counts are normalised (DESeq2 median-of-ratios), and sample structure is explored by correlation heatmap and PCA. DE is run with DESeq2 (negative-binomial model, Wald test) for each contrast, and raw p-values are converted to Storey q-values. Genes are called significant at $q < 0.05$ and $|\log2 \text{ fold change}| > 1$.

Outputs: written to the directory "1. differential_expression_analysis/"

CSV

- q1_genotype_DEG_results.csv: full Q1 DE results.
- q2_age_DEG_results.csv: full Q2 DE results.
- top50_focused_genes.csv: 50 most significant DEGs across Q1 and Q2.

Figures (PNG)

- correlation_heatmap.png: sample–sample Pearson correlation.
- pca_top500.png, pca_allgenes.png, pca_Q1_genotype.png, pca_Q2_age.png: PCA views.
- volcano_q1_genotype.png, volcano_q2_age.png: volcano plots.
- venn_overlap.png: Q1/Q2 DEG overlap.
- pvalue_distributions.png: p-value histograms (model diagnostic).
- heatmap_top_degs.png: clustered heatmap of top DEGs.
- ma_plots.png: MA plots.

Run

```
pip install pandas numpy matplotlib seaborn scikit-learn scipy pydeseq2 adjustText python differential_expression_analysis.py
```

2. q2_age_outlier_sensitivity.py

Purpose: Robustness / sensitivity check for the Q2 (age) result. Tests whether the Q2 DEG set is driven by the three technical-outlier samples flagged on the PCA (M37_WT_1, M37_WT_3, M11_WT_4) by re-running the old-vs-young DESeq2 contrast twice with all 11 wild-type samples ("full") and with the trio removed ("reduced") and comparing the two. The decisive evidence is the log2 fold-change concordance between runs, not the raw DEG count (which falls simply because dropping samples lowers power).

Input: gene_count.xls (only the wild-type age samples are used). Set path at the top of the script via DEFAULT_COUNT_MATRIX_PATH.

Method Summary: Both runs use the same filtered gene universe (≥10 reads in ≥3 samples) and the same old-vs-young DESeq2 contrast with Storey q-values ($q < 0.05$, $|\log2FC| > 1$), matching the main pipeline. The "full" run uses all 11 wild-type age samples; the "reduced" run drops the three PCA-flagged outliers (4 old + 4 young). The two are then compared on three fronts: DEG-set overlap (shared, lost, gained, Jaccard), the Pearson/Spearman correlation and sign agreement of the log2 fold changes across genes significant in either run, and whether the top-20 full-run hits remain significant after the drop. Because removing samples lowers power, the fold-change concordance is treated as the decisive robustness evidence.

Outputs: written to the directory "2. q2_age_outlier_sensitivity/"

CSV

- q2_outlier_sensitivity_comparison.csv: per-gene log2FC and q-value from both runs, with significance flags and direction agreement.
- q2_age_DEG_results_no_outliers.csv: full DESeq2 results for the reduced run.

Figures (PNG)

- q2_outlier_sensitivity.png: concordance scatter and DEG-count comparison.

Run

```
pip install pandas numpy matplotlib scipy pydeseq2 python q2_age_outlier_sensitivity.py  # or --input PATH --outdir DIR
```

3. go_enrichment_analysis.py

Purpose: Functional characterisation of the DEGs found by the DE pipeline. Runs after differential_expression_analysis.py and reads its result CSVs. Five analyses:
(1) GO/pathway enrichment with g:Profiler for the Q1/Q2 up- and down-regulated sets and the overlap
(2) transcription-factor analysis via the annotation Family column
(3) gene–gene correlation of the top DEGs
(4) detailed Q3 overlap (direction, TF family, biotype)
(5) threshold-sensitivity exploration.

Inputs: gene_count.xls (annotations + normalised expression), plus the DE pipeline outputs q1_genotype_DEG_results.csv and q2_age_DEG_results.csv. Set the paths accordingly.

Method Summary: The significant DEGs from each contrast are split by direction. Over-represented GO terms (Biological Process, Molecular Function, Cellular Component) and KEGG/Reactome pathways are identified with g:Profiler (zebrafish, FDR-corrected). DEGs are cross-referenced against the annotation Family column to flag transcription factors and summarise the families affected. The 50 most significant DEGs are correlated across all samples (Spearman) on DESeq2-normalised expression and hierarchically clustered to surface co-expressed pairs. Overlap genes are then compared by fold-change direction, and DEG counts are re-tabulated across a range of significance and fold-change thresholds to show how robust the counts are to the cut-off.

Outputs: written to the directory "3. go_enrichment_analysis/"

CSV and Figures (PNG)

- enrichment_{q1_up,q1_down,q2_up,q2_down,overlap}.csv / .png: enrichment tables and bar plots (written only when terms are found).
- q1_transcription_factors.csv, q2_transcription_factors.csv, tf_families.png: TF analysis.
- correlation_top50.png, correlation_matrix_top50.csv, correlated_gene_pairs.csv: correlation analysis.
- q3_overlap_detailed.csv, overlap_fc_scatter.png: detailed overlap.

Run (requires an internet connection for the g:Profiler step)

```
pip install pandas numpy matplotlib seaborn scipy pydeseq2 gprofiler-official adjustText python go_enrichment_analysis.py
```

4. tert_3month_de_analysis.py

Purpose: DE analysis of the 3-month-old ("young") zebrafish muscle dataset (JenAge tert_3month), the timepoint at which telomerase mutants are not yet expected to show a phenotype. It provides the young counterpart to the 18-month adult analysis. Three genotype contrasts are run: MUT vs WT, Het vs WT, and MUT vs Het and the MUT-vs-WT result is optionally compared against the 18-month genotype result.

Inputs: 13 featureCounts files (Muscle_<animalID>_fastq-featureCounts_gene.txt) in COUNTS_DIR: 5 WT, 5 Het, 3 MUT. Optionally the 18-month DE table q1_genotype_DEG_results.csv for the cross-dataset comparison (set Q1_18MONTH_CSV = ' ' to skip).

Method Summary: The 13 per-sample featureCounts tables are joined into one count matrix, low-count genes are filtered (≥10 reads in ≥3 samples), and DESeq2-normalised expression is explored by PCA (coloured by genotype). Each contrast is fitted with DESeq2 (negative-binomial model, Wald test) and assessed with Storey q-values ($q < 0.05$, $|\log2FC| > 1$), matching the main pipeline. Volcano plots are drawn per contrast, and the 3-month MUT-vs-WT DEGs are intersected with the 18-month DEGs to compare which genes are affected, and in which direction, across age.

Outputs: written to the directory "4. tert_3month_de_analysis/"

CSV and Figures (PNG)

- tert3m_raw_counts.csv: combined raw count matrix.
- tert3m_pca.png: PCA (PC1–PC2 and PC1–PC3) coloured by genotype.
- tert3m_MUT_vs_WT_results.csv, tert3m_Het_vs_WT_results.csv, tert3m_MUT_vs_Het_results.csv: DE tables.
- volcano_MUT_vs_WT.png, volcano_Het_vs_WT.png, volcano_MUT_vs_Het.png: volcano plots.
- cross_dataset_overlap_3m_vs_18m.csv: shared 3mo/18mo DEGs (only if the 18mo CSV is supplied and an overlap exists).

Run

```
pip install pandas numpy matplotlib scikit-learn scipy pydeseq2 adjustText python tert_3month_de_analysis.py
```

5. cross_dataset_direction_check.py

Purpose: Cross-dataset direction check. Takes the 5,541 genes significantly dysregulated at 18 months (Q1 genotype) and locates those same genes in the 3-month data, to test whether the age-dependent

telomerase phenotype shows any early directional signal. If the phenotype is genuinely age-dependent, the 18-month DEGs should sit centred on zero at 3 months (no early drift); an early skew in the same direction as 18 months would indicate pre-onset drift. The check is directional and distributional, not a re-run of differential expression.

Inputs: Two differential-expression result tables (gene IDs as index; a log2FoldChange column; a significance column): the 18-month Q1 genotype results q1_genotype_DEG_results.csv (output of differential_expression_analysis.py; a qvalue column is preferred, padj is used as a fallback), and the 3-month MUT-vs-WT results tert3m_MUT_vs_WT_results.csv (output of tert_3month_de_analysis.py).

Method Summary: The 18-month DEGs ($q < 0.05$, $|log2FC| > 1$) are split into up- and down-in-MUT sets and located in the 3-month table; all genes shared between the two tables form the genome-wide background. For each set, the distribution of 3-month log2 fold changes is summarised by its mean, a one-sample t-test against zero, and a Mann-Whitney U comparison against the background, and the 18-month and 3-month fold changes of the shared DEGs are correlated gene-by-gene (Pearson). A mean and correlation near zero indicate no early drift (age-dependent onset confirmed); a consistent skew would indicate early directional signal. Four figures (density, boxplot, scatter, CDF) and a per-gene summary table are written.

Outputs: written to the directory "5. cross_dataset_direction_check/"

CSV and Figures (PNG)

- cross_dataset_direction_summary.csv: per shared gene, its 3mo and 18mo log2FC, the 18mo significance value, and its 18mo direction (up/down in MUT, or not significant).
- cross_dataset_density.png: kernel-density distribution of 3mo log2FC for each 18mo gene set against the background.
- cross_dataset_boxplot.png: boxplot of 3mo log2FC by gene set, with group means annotated.
- cross_dataset_scatter.png: 18mo vs 3mo log2FC for shared DEGs, annotated with Pearson r (skipped if no shared DEGs exist).
- cross_dataset_cdf.png: cumulative distribution of 3mo log2FC per gene set.

Run

```
pip install pandas numpy matplotlib seaborn scipy python cross_dataset_direction_check.py
```

6. grn_tf_activity_model.py

Purpose: Builds a transcription-factor (TF) activity model from the Q1 (genotype) result. The 18-month sample size is too small to infer a regulatory network from the data, so a pre-built curated human network (CollecTRI) is adopted and the data is used only to score it: for each TF, are its target genes collectively up- or down-regulated in mutants? The script produces a ranked TF-activity table for Q1 and a reusable network artifact that can be re-applied to other datasets.

Inputs: Q1_CSV, the 18-month Q1 genotype DE table q1_genotype_DEG_results.csv (output of differential_expression_analysis.py), indexed by zebrafish gene ID with a gene-level statistic column ('stat', falling back to 'log2FoldChange') plus 'qvalue' and 'log2FoldChange'. A second DE table for a reuse demonstration (e.g. tert3m_MUT_vs_WT_results.csv, output of tert_3month_de_analysis.py) and an existing TF list for a sanity-check overlap (e.g. q1_transcription_factors.csv, output of go_enrichment_analysis.py). Requires an internet connection (g:Profiler orthology and the CollecTRI download via OmniPath).

Method Summary: Zebrafish gene IDs are mapped to human orthologues with g:Profiler g:Orth, and the full gene-level statistic vector (all mapped genes, not only the DEGs) is scored against the CollecTRI

regulons using decoupler's univariate linear model (ulm) to infer per-TF activity and significance. The curated network is then subset to the significant TFs (activity $p < 0.05$) and the dysregulated targets ($q < 0.05$, $|log2FC| > 1$) to give a compact, reusable GRN artifact, exported as an edge list and as GraphML (via NetworkX) for viewing in Cytoscape. An additional step re-applies the same model to a second dataset to demonstrate reuse (tert 3 month dataset).

Outputs: written to the directory "6. grn_tf_activity_model/"

CSV and Figures (PNG)

- orthology_map_zfish_to_human.csv: cached zebrafish-to-human orthologue map (reused on later runs; delete to force a re-map).
- tf_activity_Q1.csv: ranked TF activities and p-values for Q1.
- tf_activity_barplot_Q1.png: the most activated and repressed TFs.
- grn_model_edges.csv: the reusable network as an edge list (significant TFs to dysregulated targets).
- grn_model.graphml: the same network as GraphML, for opening in Cytoscape.
- tf_activity_dataset2.csv: TF activities for the second dataset.
- tf_activity_barplot_dataset2.png: the most activated and repressed TFs for the second dataset.

Run

```
pip install decoupler omnipath gprofiler-official networkx pandas numpy matplotlib python grn_tf_activity_model.py
```

7. string_prep_and_grn_overlap.py

Purpose: The STRING functional-enrichment analysis runs on the STRING website, so it is not scripted; this file scripts the two Python steps around it. Stage 1 turns the Q1 DEGs into the exact gene lists pasted into STRING (filtered to significant DEGs, split into up- and down-in-MUT, and mapped from zebrafish to human orthologues). Stage 2, once STRING's enrichment tables are downloaded, reads them back and counts how many genes in the key enriched terms are also targets of the GRN's transcription factors: the cross-validation reported in the write-up.

Inputs: Stage 1 needs the 18-month Q1 genotype DE table q1_genotype_DEG_results.csv (output of differential_expression_analysis.py) and the cached zebrafish-to-human map orthology_map_zfish_to_human.csv (output of grn_tf_activity_model.py). Stage 2 additionally needs grn_model_edges.csv and tf_activity_Q1.csv (both outputs of grn_tf_activity_model.py) and the "all enriched terms (without PubMed)" TSVs exported from STRING for the pooled, up and down runs.

Method Summary: Significant DEGs ($q < 0.05$, $|log2FC| > 1$) are split into up- and down-in-MUT sets and each zebrafish gene is mapped to its human orthologue(s); the three resulting symbol lists are written for pasting into STRING (organism = human). After the STRING runs, the downloaded enrichment tables are read back and, for the key terms (Reactome "Cell Cycle" for the down set and GO "Response to stimulus" for the up set), the member genes are intersected with the GRN's E2F / TP53 and activated / repressed transcription-factor target sets to quantify the overlap.

Outputs: written to the directory "7. string_prep_and_grn_overlap/"

CSV and txt files

- string_input_human_symbols.txt: pooled DEG list for STRING.
- string_input_UP_in_MUT_human.txt: up-in-MUT list for STRING.
- string_input_DOWN_in_MUT_human.txt: down-in-MUT list for STRING.
- grn_string_overlap_summary.csv: overlap counts between the key STRING terms and the GRN transcription-factor target sets (Stage 2 only).

Run

```
pip install pandas python string_prep_and_grn_overlap.py
```

8. q2_age_tf_activity_and_comaprison.py

Purpose: Answers the "is telomerase loss just accelerated ageing?" question the symmetric way. It scores the full CollecTRI regulon collection (about 1,185 transcription factors) against the Q2 age contrast (old 37mo versus young 11mo wild-type) from scratch, producing an independent ranked TF-activity list for natural ageing, then compares it against the full Q1 genotype TF-activity table. Every TF is scored independently in each contrast; the 71-TF exported network is not used, so the comparison is fair in both directions. Requires an internet connection (decoupler downloads CollecTRI via OmniPath).

Inputs: The Q2 age DE table q2_age_DEG_results.csv (old 37mo versus young 11mo wild-type), the orthology map orthology_map_zfish_to_human.csv, and the full Q1 TF-activity table tf_activity_Q1.csv (all as produced by the earlier programs).

Method Summary: The Q2 zebrafish genes are mapped to human orthologues (reusing the saved map) to give one gene-level statistic per human symbol; decoupler's univariate linear model scores this vector against the full CollecTRI regulons, giving an activity and p-value for every TF. The Q1 table is loaded (or optionally re-scored the same way), and the two full tables are compared by Pearson correlation, per-TF direction agreement, and the set of TFs significant in both contrasts.

Outputs: written to the directory "8. q2_age_tf_activity_and_comparison/"

CSV and Figures (PNG)

- tf_activity_age.csv: ranked TF activities and p-values for natural ageing (all scored TFs).
- q1_vs_age_tf_comparison.csv: per-TF Q1 activity, ageing activity, significance flags and direction agreement.
- q1_vs_age_scatter.png: Q1 versus ageing TF activity, with TFs significant in both contrasts labelled.

Run

```
pip install decoupler omnipath pandas numpy scipy matplotlib python
q2_age_tf_activity_and_comparison.py
```

9. interaction_genotype_x_age.py

Purpose: Tests whether the tert-/- genotype effect depends on age by combining the 3-month and 18-month datasets into a 2x2 design (genotype x age) and fitting a DESeq2 interaction model. On top of the interaction contrast it extracts every meaningful contrast from the same fitted model and builds one per-gene classification table: the genotype effect at each age, the age effect in each genotype, the interaction, the four group means, and category labels describing how each gene behaves.

Inputs: The 18-month adult-muscle raw count matrix gene_count.xls (tab separated, with gene_id, gene_name and M18_MUT/M18_WT sample columns) and the 3-month raw count matrix tert3m_raw_counts.csv (M3_MUT/M3_WT samples; heterozygotes dropped).

Method Summary: Counts from both studies are combined on shared genes, low-count genes are filtered, and a fixed-effects DESeq2 GLM (design genotype + age + genotype:age) is fitted. Five contrasts are read from the one model: the genotype effect at 3 and at 18 months, the age effect in WT and in MUT, and the genotype-by-age interaction (a difference of differences); each gets a Storey q-value. Four group means and SEMs are taken from the normalised counts. Every gene is then labelled by whether it is affected by genotype, age, both or neither ($q < 0.05$), and by how its genotype effect changes with age (age-dependent onset, transient, constitutive, reversal, or none).

Outputs: written to the directory "9. interaction_genotype_x_age/"

CSV and Figures (PNG)

- interaction_genotype_x_age_results.csv: the interaction contrast per gene.
- interaction_gene_classification.csv: the full per-gene table (group means and SEMs, all five contrasts, significance flags, and category labels).
- interaction_category_summary.csv: gene counts per category.
- interaction_lineplots.png: WT and MUT trajectory panels for the top interaction genes and tp53.
- interaction_pattern_counts.png: bar charts of the category counts.
- interaction_geno_shift_scatter.png: genotype effect at 3mo versus 18mo, coloured by interaction pattern.

Run

```
pip install pydeseq2 pandas numpy scipy matplotlib python interaction_genotype_x_age.py
```

10. grn_interaction_pattern_mapping.py

Purpose: Connects the two analyses. The GRN model found the transcription factors whose targets are dysregulated in tert-/- mutants at 18 months; the interaction model classified every gene by how its genotype effect changes with age. This program asks whether those describe the same thing, by mapping each GRN target back to its zebrafish orthologue, reading its interaction pattern, and testing whether the GRN targets are enriched for the age-dependent-onset pattern relative to the genome background. It also summarises, per transcription factor, how that factor's own targets behave.

Inputs: The interaction classification interaction_gene_classification.csv (output of interaction_genotype_x_age.py), the orthology map orthology_map_zfish_to_human.csv, the network grn_model_edges.csv, and the TF-activity table tf_activity_Q1.csv (the last three all outputs of grn_tf_activity_model.py).

Method Summary: The GRN uses human symbols and the interaction table uses zebrafish gene ids, so the two are joined through the orthology map (all zebrafish-to-human pairs kept). A zebrafish gene is called a GRN target if any of its human orthologues is a GRN target symbol; the background is every interaction gene that has a human orthologue. A Fisher exact test then compares the fraction of GRN targets that are age-dependent onset against that background, giving an odds ratio and p-value. The same genes are summarised per transcription factor and per target gene.

Outputs: written to the directory "10. grn_interaction_pattern_mapping/"

CSV and Figures (PNG)

- grn_target_interaction_map.csv: one row per GRN target gene (zebrafish), with its human symbol, how many GRN TFs regulate it, and its interaction pattern, direction and flags.
- grn_tf_pattern_summary.csv: one row per GRN TF, with the number of mapped targets, the fraction that are age-dependent onset, down in MUT and significant interaction, the TF activity, and the TF's own interaction pattern.
- grn_pattern_enrichment.png: bar charts of the interaction-pattern and affected-by distributions, GRN targets versus the genome background.

Run

```
pip install pandas numpy scipy matplotlib python grn_interaction_pattern_mapping.py
```

11. project_overview_plots.py

Purpose: Draws a small set of summary figures that give a surface-level view of the whole project: the sample design (how many fish per age and genotype), the number of significant transcription factors per contrast, and the main results across the analyses (differential expression per contrast, the interaction-model classification, and how the GRN targets map onto the age-dependent programme). Every number is computed from the source files, so the figures stay in step with the data.

Inputs: The two raw count matrices gene_count.xls and tert3m_raw_counts.csv; the DE tables q1_genotype_DEG_results.csv and q2_age_DEG_results.csv; the per-gene table interaction_gene_classification.csv; the TF-activity tables tf_activity_Q1.csv and tf_activity_age.csv; and the network files grn_model_edges.csv and orthology_map_zfish_to_human.csv.

Method Summary: Sample counts are read from the column names of the two count matrices. Significant genes are counted per contrast ($q < 0.05$ and $|\log2FC| > 1$), with the 3-month genotype and interaction counts taken from the interaction classification table. Significant transcription factors are counted per contrast and split into activated and repressed; note that TF activity is a property of a contrast (one group compared with another), not of a single group, so the counts are reported per comparison rather than per age or genotype group. The GRN target enrichment for the age-dependent-onset pattern is recomputed with a Fisher exact test against the orthologue-mapped background.

Outputs: written to the directory "11. project_overview_plots/"

Figures (PNG)

- project_sample_design.png: sample sizes per age and genotype, plus a grid showing which groups exist.
- project_tf_counts.png: significant transcription factors per contrast (genotype at 3mo, genotype at 18mo, and age), split into activated and repressed.
- project_results_overview.png: four-panel dashboard covering DEGs per contrast, the interaction pattern, the interaction classification, and the GRN target enrichment.

Run

```
pip install pandas numpy scipy matplotlib python project_overview_plots.py
```

References (code archive only)

These references support the methods and libraries used in the code only. They are separate from the report's reference list and are not intended to appear in it. Each source file numbers its own references from [1] (matching the list in that file's docstring); those per-file lists are reproduced below.

differential_expression_analysis.py

[1] M. I. Love, W. Huber, and S. Anders, "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2", *Genome Biology*, vol. 15, no. 12, art. 550, 2014.
[2] B. Muzellec, M. Telenczuk, V. Cabeli, and M. Andreux, "PyDESeq2: a python package for bulk RNA-seq differential expression analysis", *Bioinformatics*, vol. 39, no. 9, 2023.
[3] J. D. Storey and R. Tibshirani, "Statistical significance for genomewide studies", *Proc. Nat. Acad. Sci.*, vol. 100, no. 16, pp. 9440–9445, 2003.
[4] F. Pedregosa et al., "Scikit-learn: machine learning in Python", *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[5] M. L. Waskom, "seaborn: statistical data visualization", *J. Open Source Softw.*, vol. 6, no. 60, art. 3021, 2021.

q2_age_outlier_sensitivity.py

[1] M. I. Love, W. Huber, and S. Anders, "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2", *Genome Biology*, vol. 15, no. 12, art. 550, 2014.
[2] B. Muzellec, M. Telenczuk, V. Cabeli, and M. Andreux, "PyDESeq2: a python package for bulk RNA-seq differential expression analysis", *Bioinformatics*, vol. 39, no. 9, 2023.
[3] J. D. Storey and R. Tibshirani, "Statistical significance for genomewide studies", *Proc. Nat. Acad. Sci.*, vol. 100, no. 16, pp. 9440–9445, 2003.

go_enrichment_analysis.py

[1] U. Raudvere et al., "g:Profiler: a web server for functional enrichment analysis and conversions of gene lists (2019 update)", *Nucleic Acids Research*, vol. 47, no. W1, pp. W191–W198, 2019.
[2] M. I. Love, W. Huber, and S. Anders, "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2", *Genome Biology*, vol. 15, no. 12, art. 550, 2014.
[3] B. Muzellec, M. Telenczuk, V. Cabeli, and M. Andreux, "PyDESeq2: a python package for bulk RNA-seq differential expression analysis", *Bioinformatics*, vol. 39, no. 9, 2023.
[4] M. L. Waskom, "seaborn: statistical data visualization", *J. Open Source Softw.*, vol. 6, no. 60, art. 3021, 2021.

tert_3month_de_analysis.py

[1] M. I. Love, W. Huber, and S. Anders, "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2", *Genome Biology*, vol. 15, no. 12, art. 550, 2014.
[2] B. Muzellec, M. Telenczuk, V. Cabeli, and M. Andreux, "PyDESeq2: a python package for bulk RNA-seq differential expression analysis", *Bioinformatics*, vol. 39, no. 9, 2023.
[3] J. D. Storey and R. Tibshirani, "Statistical significance for genomewide studies", *Proc. Nat. Acad. Sci.*, vol. 100, no. 16, pp. 9440–9445, 2003.
[4] F. Pedregosa et al., "Scikit-learn: machine learning in Python", *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

cross_dataset_direction_check.py

[1] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nat. Methods*, vol. 17, pp. 261–272, 2020.
[2] M. L. Waskom, "seaborn: statistical data visualization", *J. Open Source Softw.*, vol. 6, no. 60, art. 3021, 2021.

grn_tf_activity_model.py

[1] U. Raudvere et al., "g:Profiler: a web server for functional enrichment analysis and conversions of gene lists (2019 update)", *Nucleic Acids Research*, vol. 47, no. W1, pp. W191–W198, 2019.

[2] P. Badia-i-Mompel et al., "decoupleR: ensemble of computational methods to infer biological activities from omics data", *Bioinformatics Advances*, vol. 2, no. 1, art. vbac016, 2022.
[3] S. Müller-Dott et al., "Expanding the coverage of regulons from high-confidence prior knowledge for accurate estimation of transcription factor activities", *Nucleic Acids Research*, vol. 51, no. 20, pp. 10934–10949, 2023.
[4] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX", in *Proc. 7th Python in Science Conf. (SciPy 2008)*, pp. 11–15, 2008.
[5] P. Shannon et al., "Cytoscape: a software environment for integrated models of biomolecular interaction networks", *Genome Research*, vol. 13, no. 11, pp. 2498–2504, 2003.

string_prep_and_grn_overlap.py
[1] D. Szklarczyk et al., "The STRING database in 2023: protein-protein association networks and functional enrichment analyses for any sequenced genome of interest", *Nucleic Acids Research*, vol. 51, no. D1, pp. D638–D646, 2023.
[2] U. Raudvere et al., "g:Profiler: a web server for functional enrichment analysis and conversions of gene lists (2019 update)", *Nucleic Acids Research*, vol. 47, no. W1, pp. W191–W198, 2019.

q2_age_tf_activity_and_comparison.py
[1] S. Müller-Dott et al., "Expanding the coverage of regulons from high-confidence prior knowledge for accurate estimation of transcription factor activities", *Nucleic Acids Research*, vol. 51, no. 20, pp. 10934–10949, 2023.
[2] P. Badia-i-Mompel et al., "decoupleR: ensemble of computational methods to infer biological activities from omics data", *Bioinformatics Advances*, vol. 2, no. 1, art. vbac016, 2022.
[3] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods*, vol. 17, pp. 261–272, 2020.

interaction_genotype_x_age.py
[1] M. I. Love, W. Huber, and S. Anders, "Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2", *Genome Biology*, vol. 15, no. 12, art. 550, 2014.
[2] B. Muzellec, M. Telenczuk, V. Cabeli, and M. Andreux, "PyDESeq2: a python package for bulk RNA-seq differential expression analysis", *Bioinformatics*, vol. 39, no. 9, 2023.
[3] J. D. Storey and R. Tibshirani, "Statistical significance for genomewide studies", *Proc. Nat. Acad. Sci.*, vol. 100, no. 16, pp. 9440–9445, 2003.

grn_interaction_pattern_mapping.py
[1] S. Müller-Dott et al., "Expanding the coverage of regulons from high-confidence prior knowledge for accurate estimation of transcription factor activities", *Nucleic Acids Research*, vol. 51, no. 20, pp. 10934–10949, 2023.
[2] P. Badia-i-Mompel et al., "decoupleR: ensemble of computational methods to infer biological activities from omics data", *Bioinformatics Advances*, vol. 2, no. 1, art. vbac016, 2022.
[3] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods*, vol. 17, pp. 261–272, 2020.

project_overview_plots.py
[1] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods*, vol. 17, pp. 261–272, 2020.

Dependencies
Python 3.10+ with: pandas, numpy, matplotlib, seaborn, scikitlearn, scipy, pydeseq2, adjustText, gprofiler-official, decoupler, omnipath, networkx.